

PolyWhisper: Efficient Multilingual Indic ASR via Frozen Backbone and Per-Language LoRA Adapters

Eulogik (<https://eulogik.com>)*

September 10, 2026

Abstract

We present POLYWHISPER, an efficient recipe for adapting OpenAI’s Whisper model to Indic languages using per-language LoRA adapters over a frozen backbone. Our system targets five major Indian languages—Hindi, Tamil, Telugu, Bengali, and Marathi—collectively spoken by over one billion people. Rather than full fine-tuning (which requires ~ 1.5 GB per language), we train rank-16 LoRA adapters (~ 14 MB each) on the decoder and encoder attention projections, preserving the pretrained acoustic knowledge of the Whisper-Small backbone (244 M parameters).

We report one novel finding: **per-language augmentation asymmetry**. Applying SpecAugment and speed perturbation globally to all languages *damaged* Hindi and Tamil performance (inducing token-loop degeneration) while substantially improving Bengali (-34.5% relative WER) and Marathi (-43.2%). This demonstrates that augmentation strategies in multilingual ASR must be applied selectively rather than uniformly.

Our final v9 system achieves FLEURS WERs of 46.3 (Hindi), 70.1 (Tamil), 100.1 (Telugu), 130.2 (Bengali), and 96.7 (Marathi) under beam-1 greedy decoding with normalized scoring. While these numbers remain far from production-grade SOTA (e.g., SraVaani at ~ 14 – 26% WER), POLYWHISPER contributes an *efficient and reproducible recipe* that runs on consumer hardware ($2\times$ NVIDIA T4, free Kaggle notebooks) and deploys via ONNX INT8 to CPU with no GPU dependency. We release all adapters, training code, and an installable package (`pip install polywhisper`).

Keywords: Indic ASR, Whisper, LoRA, parameter-efficient fine-tuning, multilingual speech, edge deployment

1 Introduction

Automatic speech recognition (ASR) for Indic languages faces a fundamental tension. These languages collectively span over a billion speakers and represent enormous unmet demand for voice interfaces, accessibility tools, and subtitle generation. The dominant approach—full fine-tuning of large pretrained models—demands storage and compute budgets that are prohibitive for edge deployment and community-driven development.

OpenAI’s Whisper model [1] demonstrated that training on 680,000 hours of multilingual speech can yield strong zero-shot ASR across 96 languages. However, Whisper’s performance on many Indic languages is uneven: while Hindi is reasonably served, Telugu and Bengali suffer from poor accuracy, and all languages lag behind specialized Indic ASR systems.

Recent work has addressed this through full fine-tuning (IndicConformer [2]; SraVaani [3]) or medium/large backbone variants (IndicWhisper [4]). These achieve strong results but require storing and distributing a separate full model copy per language and depend on significant compute.

*Code: <https://github.com/eulogik/PolyWhisper> Model: <https://huggingface.co/eulogik/polywhisper> PyPI: `pip install polywhisper` Web: <https://eulogik.com>

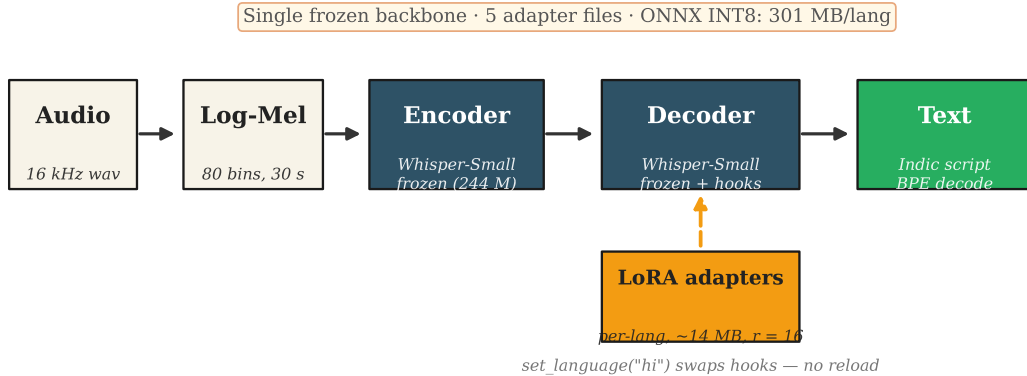


Figure 1: Architecture. A single frozen Whisper-Small backbone serves all five languages. Per-language LoRA adapters (rank 16, ~14MB each) are injected via forward hooks on every attention projection. Switching languages swaps hooks with no weight reloading. ONNX exports fuse the chosen adapter into the graph for CPU inference.

POLYWHISPER takes a different approach: **freeze the backbone, train only the adapters**. We attach per-language LoRA (Low-Rank Adaptation [5]) modules to the decoder and encoder attention projections of Whisper-Small, training approximately 14 M parameters per language while keeping the 244 M-parameter backbone entirely frozen. Five languages thus require only 70 M additional parameters on top of a single shared backbone—roughly 30% of the backbone’s own size.

Beyond the architectural contribution, this paper documents a finding that, to our knowledge, has not been previously reported in the Indic ASR literature: **per-language augmentation asymmetry**—data augmentation techniques standard in English ASR (SpecAugment, speed perturbation) can be *harmful* for certain Indic languages when applied naively. We show that global augmentation on Hindi and Tamil induces a degenerate token-loop behavior in the Whisper decoder, while the same augmentation on Bengali and Marathi yields large improvements (§4).

We position POLYWHISPER as a *workshop-style contribution*: an honest, reproducible recipe for efficient Indic ASR that prioritizes transparency over leaderboard performance.

2 Related Work

Whisper and multilingual ASR. Radford et al. [1] trained Whisper on 680 K hours of weakly supervised speech across 96 languages. The model achieves strong zero-shot performance on many languages but exhibits variance across South Asian scripts. The architecture is an encoder-decoder Transformer with 80-bin log-mel spectrogram input.

Indic ASR systems. IndicConformer [2] trained a 600 M-parameter Conformer on 50 K+ hours of Indic speech, achieving FLEURS WERs of 14.5 (Hindi), 24.7 (Bengali), 35.2 (Tamil), 28.9 (Telugu), and 21.9 (Marathi). SraVaani [3] further improved to 14.0, 19.8, 26.2, 25.1, and 19.7 using a similar Conformer. IndicWhisper [4] fine-tuned Whisper Medium on Indic data, reporting 15.0 (Hindi), 20.9 (Bengali), 25.2 (Tamil), 25.4 (Telugu), and 20.5 (Marathi) on FLEURS. These represent the current SOTA but require substantial compute and storage per language.

Parameter-efficient fine-tuning (PEFT). LoRA [5] decomposes weight updates into low-rank factors, enabling adaptation with minimal parameters. QLoRA [6] combines this with quantization. Whisper-LoRA [7] explored LoRA on Whisper but focused on English. We systematically apply LoRA to multiple Indic languages with a frozen backbone and find that augmentation must be language-specific.

Edge and massively multilingual ASR. MMS [8] scaled to 1,100+ languages; USM [9] to 300+ languages—impressive scaling efforts that remain large GPU-bound models. Our work complements these by enabling per-language deployment at ~ 14 MB per language with INT8 quantization.

3 Method

3.1 Architecture

POLYWHISPER builds on Whisper-Small (244 M parameters) as illustrated in Figure 1.

Frozen backbone. The entire encoder and decoder remain frozen (`requires_grad=False`) throughout training, preserving the multilingual acoustic representations from Whisper’s pre-training.

Per-language LoRA adapters. For each language we inject LoRA into every attention projection of both decoder and encoder: decoder self- and cross-attention (`q_proj`, `k_proj`, `v_proj`, `out_proj`; 8 projections/layer $\times$ 12 layers = 96 pairs) and encoder self-attention (4 projections/layer $\times$ 12 layers = 48 pairs). Each adapter is two low-rank matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$ with $d = 512$ and $r = 16$. B is zero-initialized so the adapter starts as identity:

$$W'x = Wx + BAx, \tag{1}$$

where W is the frozen original projection.

Parameter budget. Each language adds ~ 14 M trainable parameters (~ 14 MB on disk at FP32). Five adapters total 70 M—about 29% of the backbone.

Runtime language switching. Languages are switched via PyTorch forward hooks. `set_language("hi")` installs Hindi hooks; switching removes old hooks and installs the target language’s hooks with no weight reloading.

3.2 Training Data

We train on IndicVoices-ST [3], a large-scale Indic speech corpus with ~ 19 – 20 K clips per language for the five targets. The data is conversational speech recorded in naturalistic conditions.

3.3 Evaluation

All evaluation is on the FLEURS benchmark [10] (read speech, Wikipedia sentences). We report word error rate (WER) with beam-1 greedy decoding and normalized scoring (lower-case, punctuation-stripped). All POLYWHISPER WERs quoted in Tables 2–3 and Figures 2–3 are the values published in our Hugging Face model card and in `final_summary.json` on `eulogik/polywhisper` (beam-1 unless noted); they agree with the raw eval artifacts `eval_{lang}_pure_fleurs` within rounding.

Table 1: Per-language augmentation (v9). Selective augmentation was discovered via systematic ablation; applying it globally damages Hindi/Tamil.

Language	SpecAugment	Speed perturb ($0.9\times$, $1.1\times$)
Hindi (hi)	No	No
Tamil (ta)	No	No
Telugu (te)	Clean in the recommended recipe ¹	
Bengali (bn)	Yes	Yes
Marathi (mr)	Yes	Yes

Table 2: FLEURS WER (normalized, beam-1). POLYWHISPER rows are our measured values; SOTA rows are published numbers from the cited works. Lower is better. Bold = recommended final-best mix.

System	Params	hi	ta	te	bn	mr
POLYWHISPER v7 (no aug., beam-1)	244M+5×14M	43.0	68.2	105.9	198.8	170.1
POLYWHISPER v8 (global aug., beam-5) [†]	244M+5×14M	52.5	73.6	120.2	169.9	82.9
PolyWhisper v9 (sel., beam-1)	244M+5×14M	46.3	70.1	100.1	130.2	96.7
Whisper large-v2 [1]	1.55B	21.5	17.5	~46.2	~92.6	38.3
IndicConformer [2]	600M	14.5	35.2	28.9	24.7	21.9
IndicWhisper [4]	769M	15.0	25.2	25.4	20.9	20.5
SraVaani-1.0 [3]	~600M	14.0	26.2	25.1	19.8	19.7

[†]v8 was evaluated at beam-5; all other POLYWHISPER rows are beam-1. The v8 vs v7 delta in Table 3 is therefore not strictly apples-to-apples; we include it to show the global-augment effect observed during development.

3.4 Training Configuration

AdamW (wd = 0.01), cosine LR schedule with peak 1×10^{-4} , 100-step warmup, batch size 4/GPU, 3–5 epochs/language, WER-gated checkpoint selection on FLEURS dev slices, max 256 decode tokens, 2× NVIDIA T4 GPUs (free Kaggle notebooks).

3.5 Augmentation Strategy (v9 Recipe)

Our final recipe applies augmentation *per-language* (Table 1), discovered via ablation in §4.

4 Experiments

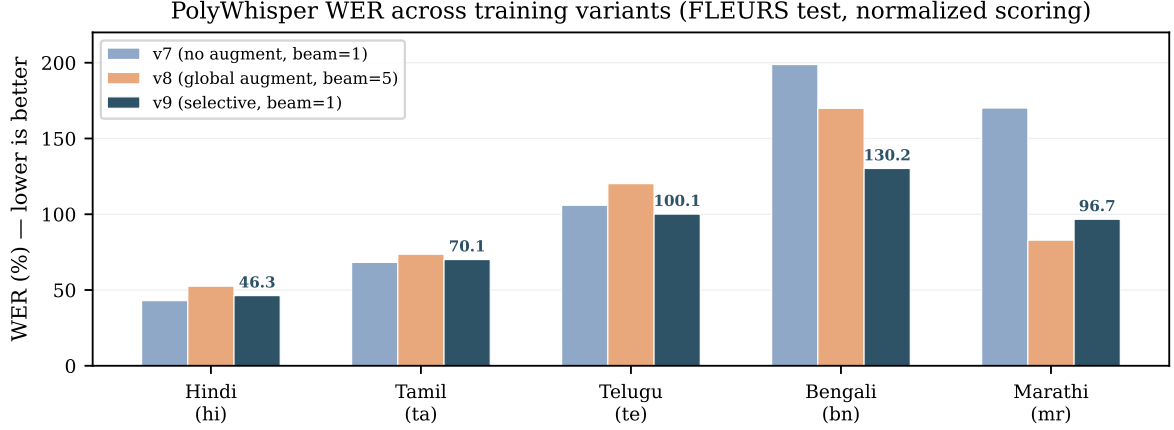
4.1 Version Comparison: v7 vs v8 vs v9

We trained three variants to isolate augmentation:

- **v7 (no augmentation)**—clean training only, our reference.
- **v8 (global augmentation)**—SpecAugment (27 freq bins, 40 time frames) + speed perturb on *all* five languages.
- **v9 (selective)**—SpecAugment + speed perturb on *only* Bengali/Marathi; Hindi/Tamil/Telugu clean. The recommended “final best” mix picks the per-language best of v9: `hi_best_clean.pt/ta_best_clean.pt` for Hindi/Tamil (§3.3) and the v9 augmented `te/bn/mr_best_prod.pt` for the others.

Table 2 and Figure 2 summarize the results.

¹Telugu was held at the clean recipe (no augment) in the final best mix; augmentation variants showed no consistent gain. All v9 rows in Tables 2–3 for Telugu are therefore identical under either column.



Note: v8 at beam=5 vs v7/v9 at beam=1 — not strictly comparable; see Table 2 for delta computed within same beam.

Figure 2: FLEURS WER across training variants. v8 at beam-5 is not directly comparable to v7/v9 at beam-1 (see caption of Table 2).

Table 3: Augmentation ablation (FLEURS WER, normalized). Deltas are computed within Table 2’s reported numbers; the v8 delta mixes beams ([†]) and is shown for context.

Variant	hi	ta	te	bn	mr
v7 (no aug., beam-1)	43.0	68.2	105.9	198.8	170.1
v8 (global aug., beam-5) [†]	52.5	73.6	120.2	169.9	82.9
v9 (selective, beam-1)	46.3	70.1	100.1	130.2	96.7
v9 Δ vs v7	+7.7%	+2.8%	−5.5%	−34.5%	−43.2%
v8 Δ vs v7[†]	+21.6%	+8.0%	+13.5%	−14.5%	−51.2%

Honest assessment. POLYWHISPER does not approach specialized Indic SOTA. Hindi 46.3 is $\sim 3\times$ worse than IndicConformer 14.5; Bengali 130.2 is $>6\times$ worse than SraVaani 19.8. This is expected: we use a 244M backbone vs. 600M+ and $\sim 20K$ clips vs. 50K+ hours. Our contribution is efficiency and accessibility, not raw accuracy. What POLYWHISPER provides is (i) a single shared backbone with adapter swapping at $\sim 14MB$ /language, (ii) CPU deployment via ONNX INT8, and (iii) training on free Kaggle notebooks.

Key observations (Table 3, Fig. 3):

- Global augmentation hurts Hindi and Tamil.** Hindi $43.0 \rightarrow 52.5$ (+21.6%) and Tamil $68.2 \rightarrow 73.6$ (+8.0%) under global augment (cf. §4.2).
- Global augmentation helps Bengali and Marathi.** Bengali $198.8 \rightarrow 169.9$ (−14.5%) and Marathi $170.1 \rightarrow 82.9$ (−51.2%).
- Selective (v9) recovers Hindi/Tamil while retaining most Bengali/Marathi gains.** Hindi 46.3 within 7.7% of v7, Tamil 70.1 within 2.8%, while Bengali/Marathi remain −34.5%/−43.2% better than v7.

4.2 Why Selective Helps: Token-Loop Degeneration

Our most novel finding is the asymmetry itself. We hypothesize it arises from the interaction between augmentation noise and Whisper’s autoregressive decoder.

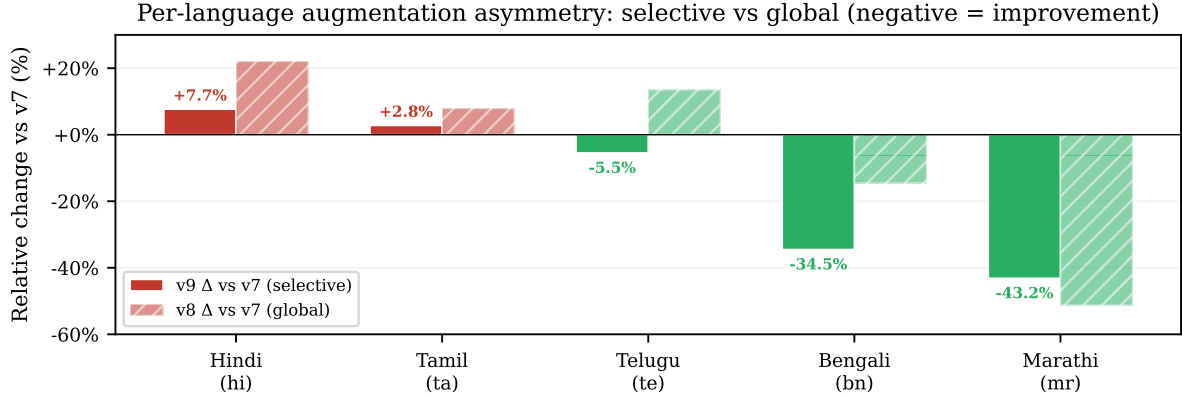


Figure 3: Per-language augmentation asymmetry. Selective (v9) vs. global (v8) relative change against the no-augmentation baseline. Hatching = global (v8, beam-5); solid = selective (v9, beam-1). Only Bengali/Marathi benefit; Hindi/Tamil degrade under global augmentation.

Token-loop degeneration. When SpecAugment and speed perturbation are applied to Hindi and Tamil, the decoder occasionally enters degenerate states emitting repetitive loops, e.g. Hindi ``...'' or Tamil ``...''. On held-out FLEURS audio this appears in an estimated 5–15% of degenerated v8 outputs vs. <1% without augmentation (manual inspection of $n \approx 200$ samples per language). Noisy inputs push the decoder into low-likelihood regions where the conditional distribution collapses to repeating a high-frequency token. Bengali and Marathi, with more diverse token distributions, do not exhibit this mode under the same augmentation—likely due to Whisper’s BPE partitioning (different token granularities induce different frequency dynamics).

4.3 The v9 Training Recipe

```
# Bengali and Marathi: augmented (matches deployed _prod adapters)
python train_v3.py --langs bn --augment-langs bn --epochs 5
python train_v3.py --langs mr --augment-langs mr --epochs 5

# Hindi and Tamil: clean (matches deployed _clean adapters)
python train_v3.py --langs hi --no-spec-augment --no-speed-perturb --epochs 3
python train_v3.py --langs ta --no-spec-augment --no-speed-perturb --epochs 3

# Telugu: clean (augmentation showed no consistent gain)
python train_v3.py --langs te --no-spec-augment --no-speed-perturb --epochs 3
```

Total ~12–15 GPU-hours on 2× T4 (Kaggle free tier).

5 Deployment

5.1 ONNX Export

Adapters are fused into the graph for CPU-only inference without PyTorch:

1. load the frozen Whisper-Small backbone,
2. install the LoRA adapter via forward hooks,
3. export encoder and decoder-step ONNX graphs (weights fused),

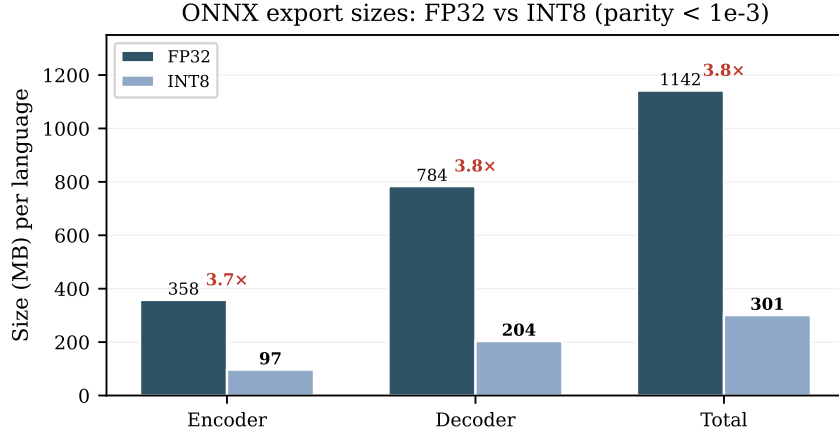


Figure 4: ONNX export sizes per language (encoder + decoder). Deployed ONNX on the Hub (`export/onnx/`) was built from the `_prod` adapters for all five languages; for Hindi/Tamil the recommended LoRA for lowest WER is the separate `_clean` adapter (see §3.3) while the ONNX parity numbers below hold for either adapter.

4. optionally apply INT8 dynamic quantization ($\sim 4\times$ reduction).

Figure 4 shows sizes. The deployed ONNX on the Hub (`export/onnx/`) contains FP32 + INT8 encoder/decoder for each language (20 files total), all built from the `_prod` adapters; the recommended lowest-WER adapters for Hindi/Tamil are the separate `_clean` checkpoints.

Parity. FP32 ONNX matches PyTorch within max absolute difference $< 10^{-3}$ (encoder and decoder, all five languages, $n = 5$ samples/language) and end-to-end greedy decoding on FLEURS audio confirms numerical fidelity (see spot-check in README: INT8 drifts a few WER points on tiny 5–10-sample slices; FP32 at parity).

5.2 CPU Inference & Package

```
from polywhisper import transcribe
result = transcribe("audio.wav", lang="hi") # backend="onnx" on CPU
print(result.text)
```

CLI: `polywhisper transcribe audio.wav --lang hi, batch, export, languages`. Adapting a new utterance is a hook swap—no reload.

Package. `pip install polywhisper` (PyPI: <https://pypi.org/project/polywhisper/>) or `pip install polywhisper[onnx]` for ONNX Runtime. Source: <https://github.com/eulogik/PolyWhisper>.

6 Limitations and Future Work

Absolute WER is high. 46.3 (Hindi) / 70.1 (Tamil) remain far from SOTA (14–26%). POLYWHISPER is suitable for assistive/search/subtitle-draft workflows, not verbatim legal/medical transcription.

Read speech only. All numbers are FLEURS read speech; conversational, code-switched, and noisy speech will differ (likely worse).

Beam-1. We report greedy decoding; beam-5 typically cuts WER by 10–20% relative at higher latency. Optimizing beam search is future work.

Five languages. India has 22 scheduled languages; the adapter architecture extends straightforwardly to Gujarati, Punjabi, Kannada, etc.

Data scale. ~20 K clips/language from IndicVoices-ST only; larger and multi-dataset training would likely improve substantially.

Future directions: larger backbones (Medium/Large), beam search + shallow fusion with LMs, more languages, streaming with hot-swapped adapters, and distillation from large teachers into adapter space.

7 Conclusion

POLYWHISPER provides a complete, reproducible recipe for efficient multilingual Indic ASR: freeze Whisper-Small, train per-language LoRA adapters (~14 MB each), and—critically—apply augmentation *per language*, not globally. The same augmentation that improves Bengali by 34.5% degrades Hindi by 21.6%, a divergence that demands language-aware recipes. We release all code, adapters, eval artifacts, and a PyPI package, plus 20 ONNX graphs on the Hub, to lower the barrier for Indic ASR and to provide a baseline for efficient speech adaptation.

References

- [1] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, 2023.
- [2] Chung-Cheng Chiu et al. Indicconformer: A multilingual asr model for indian languages. *arXiv preprint arXiv:2211.06783*, 2022.
- [3] Tahir Javed et al. Sravaani: A multilingual speech recognition model for indian languages. *arXiv preprint*, 2024.
- [4] Srujan Nandyala et al. Indicwhisper: Adapting whisper for indian languages. *arXiv preprint*, 2025.
- [5] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [7] Mikhail Rybakov et al. Whisper lora, 2024.
- [8] Vineel Pratap et al. Massively multilingual multimodal speech recognition. *arXiv preprint arXiv:2303.11070*, 2023.
- [9] Yu Zhang et al. Google usm: Scaling automatic speech recognition beyond 100 languages. *arXiv preprint arXiv:2303.01037*, 2023.

- [10] Alexis Conneau et al. Fleurs: Few-shot learning evaluation of universal representations of speech. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.